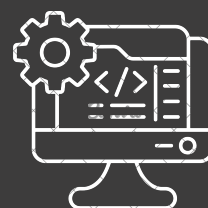


Desenvolvimento de um Simulador de Escalonamento e Shell Interativo

Linguagem C, POSIX Threads,
Mutexes e Variáveis de Condição

Arquitetura Do Sistema



O mini-bash (imesh):

Mini bash gerindo processos via `fork()` e `execvp()`, com suporte a comandos como `pwd`, `date` e `kill`;



O Simulador (ep1):

Um programa independente invocado pelo shell para simular processos de um arquivo de trace.



Algoritmos:

Para este EP foram implementados escalonadores SJF, RR e Priority Scheduler

Modelagem de Processo com Threads

Estrutura do Processo:

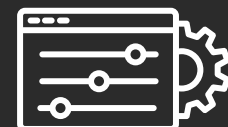
Cada linha do arquivo de trace vira uma `struct` Processo e uma `pthread_t` correspondente.

O Campo `dt` (Duration Time):

Representa a carga de trabalho total. A thread só encerra quando `tempo_restante` chega a zero.

Sincronização:

Uso de `pthread_mutex_t` para proteger dados compartilhados e `pthread_cond_t` para o controle de execução (ping-pong entre Escalonador e Threads).



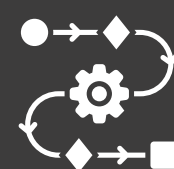
Escalonador 1

Shortest Job First (SJF)



Lógica:

Algoritmo não-preemptivo que prioriza processos com menor **dt.**



Implementação:

Fila de prontos ordenada por tempo de execução.



Vantagem:

Minimiza o tempo médio de espera ao evitar que processos curtos fiquem presos atrás de processos longos.

Escalonador 2

Round Robin (RR)

Lógica:

Algoritmo preemptivo baseado em fatias de tempo fixas (Quantum).

Preempção:

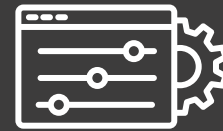
Quando o quantum acaba, o processo volta para o fim da fila e o contador de preempções é incrementado.

Justiça:

Garante que todos os processos recebam atenção da CPU periodicamente.

Escalonador 3

Prioridade (Deadline)



Estratégia:

Earliest Deadline First (EDF).



Diferencial:

Quantum dinâmico baseado na proximidade do deadline e margem de segurança.



Objetivo:

Minimizar o número de processos que falham em cumprir seus prazos.



Desafios de Sincronização

Busy-Waiting vs. Sleep:

Substituição do `sleep()` por loops de espera ativa (`clock_gettime`) para simular carga real de CPU em arquiteturas modernas (como o Apple M1).

Evitando Deadlocks:

Garantia de que o Mutex seja liberado durante o `pthread_cond_wait` para que a thread possa progredir.

O Experimento

"Esperado"



Cenário:

Mistura de processos curtos e longos chegando em tempos próximos.



Resultado:

O SJF apresenta o melhor tempo de espera médio, enquanto o RR tem mais preempções e maior overhead.

O Experimento "Inesperado"



Cenário (Efeito Comboio):

Um processo enorme chega primeiro ($t_0=0$), seguido de vários processos minúsculos.



Inversão:

O RR se mostra superior ao SJF para a interatividade, pois os processos pequenos não precisam esperar o gigante terminar.

The background of the slide is an abstract, artistic image featuring swirling, ethereal smoke or ink in various shades of green and purple. The colors are vibrant and layered, creating a sense of depth and movement. On the right side, there are thin, light green curved lines that sweep across the white background, adding a modern, graphic touch to the design.

Bibliografia

Compatibilidade:

Para este trabalho foram consultados as Manpages do linux, stackoverflow, a playlist sobre Sistemas Operacionais do canal Corde Dumped e uma pequena conversa com o Gemini para gerar os arquivos de trace e corrigir bugs de ponteiros (<https://gemini.google.com/share/e59acaf7eade>)

Conclusão

- O simulador cumpre os requisitos de concorrência e escalonamento.
- A escolha do EDF para prioridade provou ser eficaz para sistemas de tempo real.
- Aprendizado sobre o custo de troca de contexto e a importância da sincronização robusta.